OpenFace 3.0 Reference

What you can call from the installed **openface** package, what each call returns, and what the models were trained on. Written for the AffectProject setup (pip package **openface-test** in .venv, weights in OpenFace-3.0/weights).

The pipeline at a glance

Everything comes from three classes. A typical frame goes through them in this order:

Step	Class	Input	Output
1	`FaceDetector`	Image (file path; or a frame via your ArrayFaceDetector)	Face bounding box + confidence + 5 facial points; a cropped face image
2a	`MultitaskPredictor`	Cropped face image	8 emotion scores, 2 gaze angles, 8 Action Unit scores
2b	`LandmarkDetector`	Full image + detections from step 1	98 (or 68) landmark points per face; input for PnP head pose

The code you import lives in .venv/lib/python3.11/site-packages/openface/, not in the OpenFace-3.0 folder. The OpenFace-3.0 folder supplies the weights, sample images, and reference source.

1. FaceDetector

```
from openface.face_detection import FaceDetector

det = FaceDetector(model_path=str(W / "Alignment_RetinaFace.pth"), device="cpu",
                   confidence_threshold=0.02, nms_threshold=0.4, vis_threshold=0.5)
```

Method	Returns	Notes
`get_face(image_path, resize=1.0)`	`(face_crop, dets)` or `(None, None)`	Crop of the top-scoring face. Returns None if no face scores above vis_threshold.
`detect_faces(image_path, resize=1.0)`	`(dets, img_raw)`	Every face found, after non-max suppression, sorted by score.

dets format: one row per face, 15 numbers: `[x1, y1, x2, y2, score, x_l_eye, y_l_eye, x_r_eye, y_r_eye, x_nose, y_nose, x_l_mouth, y_l_mouth, x_r_mouth, y_r_mouth]`. The first four values are the box corners in pixels.

Parameters: confidence_threshold drops weak candidate boxes early; nms_threshold merges overlapping boxes; vis_threshold is the minimum score get_face() accepts.

Gotchas. (1) The shipped preprocess_image() only accepts a file path; for webcam or video frames use the ArrayFaceDetector subclass in src/live_demo.py. (2) Loading the model also reads ./weights/mobilenetV1X0.25_pretrain.tar by relative path, so os.chdir() into OpenFace-3.0 first. That file comes from the git clone, not the weight download script.

2. MultitaskPredictor (all the affect outputs)

```
from openface.multitask_model import MultitaskPredictor

mtl = MultitaskPredictor(model_path=str(W / "MTL_backbone.pth"), device="cpu")
emotion, gaze, aus = mtl.predict(face_crop)   # face_crop: BGR numpy array
# shapes: emotion (1, 8)   gaze (1, 2)   aus (1, 8)
```

One EfficientNet-B0 network with three output heads. The face is resized to 224x224 internally. Each head was trained on a different dataset:

Emotion head: 8 classes, trained on AffectNet

AffectNet is about 1M web photos with human-labelled expressions. The output is 8 raw scores (logits); use `torch.softmax(emotion, dim=1)` for probabilities. The label set is fixed; this head cannot output any other category.

Index	0	1	2	3	4	5	6	7
Emotion	Neutral	Happy	Sad	Surprise	Fear	Disgust	Anger	Contempt

Tip: the full 8-value probability vector is a richer LSTM feature than the single top label that live_demo.py displays.

Gaze head: 2 values, trained on MPIIGaze

MPIIGaze is laptop-webcam images of people. The output is `[yaw, pitch]`: yaw is left/right, pitch is up/down. The package doesn't document the units; the MPIIGaze convention is radians. Calibrate by looking at the screen center and its edges. Useful for an AI tutor as a looking-at-screen signal.

Action Unit head: 8 values, trained on DISFA

Action Units (AUs) are facial muscle movements from the Facial Action Coding System (FACS). DISFA labels 12 AUs; training keeps 8, in this output order:

Index	AU	Muscle movement	Typical meaning
0	AU1	Inner brow raiser	Surprise, worry, concentration
1	AU6	Cheek raiser	Genuine (Duchenne) smile
2	AU17	Chin raiser	Doubt, pouting
3	AU25	Lips part	Mouth open, talking
4	AU26	Jaw drop	Surprise, yawning
5	AU2	Outer brow raiser	Surprise
6	AU12	Lip corner puller	Smiling
7	AU15	Lip corner depressor	Sadness, disapproval

Mapping is probable, not confirmed. It is derived from `eval_au = [0, 4, 8, 10, 11, 1, 6, 7]` in OpenFace-3.0/train_mix_au_dev.py, line 121, assuming DISFA's standard AU order (the label files aren't in the repo). Check: smile at the webcam; indices 6 (AU12) and 1 (AU6) should rise.

Reading the values. These are *not* intensities, despite the README. Training turned DISFA's 0-5 intensity labels into present/absent (present = intensity 2 or higher), and the head outputs a cosine-similarity score in [-1, 1]. Training treats a score **> 0.5** as present. So `#6:0.71` in the live demo means "AU12 (smile) is probably active", not "71% intensity".

3. LandmarkDetector

```
from openface.landmark_detection import LandmarkDetector

lmk = LandmarkDetector(model_path=str(W / "Landmark_98.pkl"), device="cpu")
dets, img = det.detect_faces(image_path)
points = lmk.detect_landmarks(img, dets, confidence_threshold=0.5)
```

```
# list with one array of (x, y) points per face that passed the threshold
```

Uses the STAR alignment model. Landmark_98.pkl gives the 98-point WFLW layout (the indices src/visualize_frame.py already uses); Landmark_68.pkl gives the classic 68-point layout. These points are what you'd feed into PnP (cv2.solvePnP with a 3D face model) to get head pose (yaw/pitch/roll), which OpenFace 3.0 does not output itself.

> **Gotcha.** Requires scripts/patch_openface.sh. Without it, STAR's config points at the authors' /work/... Docker path and crashes on macOS. Re-run the patch after every venv rebuild.

Command-line interface

Command	What it does	Status on your Mac
`openface detect IMG --device cpu -o DIR`	Runs all three models on one image, writes a CSV of landmarks, gaze, AUs and emotion	Works; run from inside OpenFace-3.0
`openface detect-video VID ...`	Per-frame CSV for a video	Broken (passes frames into a path-only API); write your own loop
`openface download`	Fetches weights from HuggingFace	Skips if weights/ exists; use scripts/download_weights.sh

Always pass --device cpu: the CLI defaults to cuda, which Macs don't have.

What OpenFace 3.0 does not give you

Not provided	How the project gets it
Engagement, boredom, confusion, frustration (the DAiSEE labels)	Your LSTM learns these from sequences of per-frame OpenFace features
Head pose (yaw / pitch / roll)	PnP on the 98 landmarks, then Kalman smoothing
Valence / arousal	Not available; AffectNet has these labels but this model wasn't trained on them
AU intensity (0-5)	Not available; only present/absent scores for 8 AUs
Any emotion beyond the 8 classes	Not available; the label set is fixed

Minimal per-frame feature extraction

```
import os, torch
from pathlib import Path
from openface.multitask_model import MultitaskPredictor

PROJECT_ROOT = Path(__file__).resolve().parents[1]
W = PROJECT_ROOT / "OpenFace-3.0" / "weights"
os.chdir(W.parent)                    # RetinaFace loads ./weights/... by relative path

det = ArrayFaceDetector(model_path=str(W / "Alignment_RetinaFace.pth"), device="cpu")
mtl = MultitaskPredictor(model_path=str(W / "MTL_backbone.pth"), device="cpu")

def frame_features(frame):            # frame: BGR numpy array
    face, dets = det.get_face(frame)
    if face is None:
        return None
```

```
    emo, gaze, aus = mtl.predict(face)
    return torch.cat([torch.softmax(emo, 1), gaze, aus], dim=1).squeeze(0)  # 18 values
```

ArrayFaceDetector is the subclass defined in src/live_demo.py. 18 values = 8 emotion probabilities + 2 gaze + 8 AU scores; add PnP head pose for the full feature vector.

Sources

Installed package: .venv/lib/python3.11/site-packages/openface/ (face_detection.py, multitask_model.py, landmark_detection.py, model/MTL.py, model/AU_model.py, cli.py). OpenFace-3.0/README.md (API section) and train_mix_au_dev.py (datasets, AU subset, 0.5 threshold). Paper: Hu, Mathur, Liang & Morency, "OpenFace 3.0: A Lightweight Multitask System for Comprehensive Facial Behavior Analysis", arXiv:2506.02891 (2025).